Sprint 1 - pop quiz report

Prepared by: Aicha Nait Dayaa

Date: February 15, 2026

1 Chapter 1: Backend Challenges

1.1 Error Handling :

- **Problem :**

  The page ***http://localhost:8888/broken*** returns a 500 Internal Server Error without any indication of the cause.

- **Investigation :**

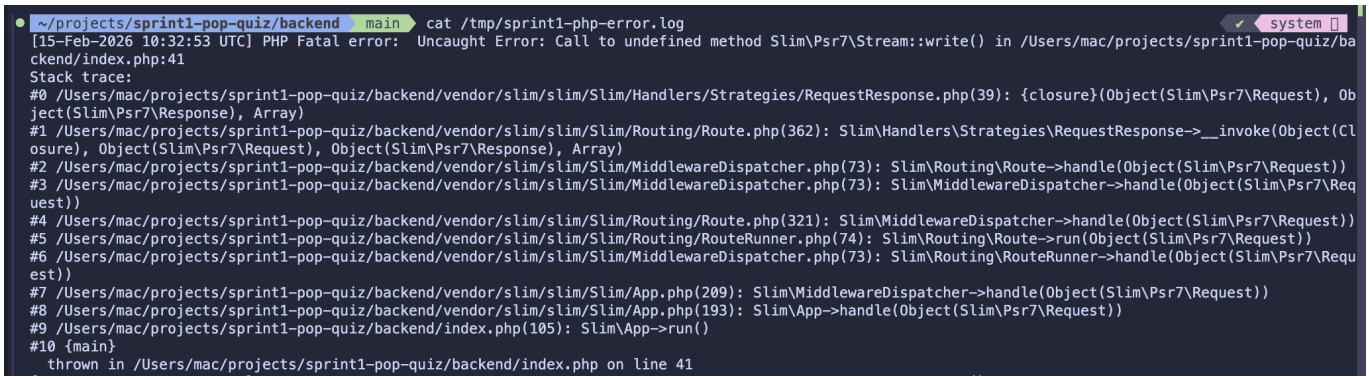
  This block of code in the `index.php` file :

```
error_reporting(E_ALL);
ini_set('log_errors', 1);
ini_set('error_log', '/tmp/sprint1-php-error.log');
ini_set('display_errors', 0);
```

  is responsible for handling error reporting and logging:

  - `error_reporting(E_ALL);` enables reporting of all types of PHP errors, including warnings and fatal errors.
  - `ini_set('log_errors', 1);` allows errors to be saved to a log file.
  - `ini_set('error_log', '/tmp/sprint1-php-error.log');` specifies the path of the error log file.
  - `ini_set('display_errors', 0);` disables showing errors in the browser, keeping the interface clean for users.

  Since the endpoint `/broken` returned a 500 Internal Server Error with no visible message, we checked the log file at `/tmp/sprint1-php-error.log` and found the following entry:



```
~/projects/sprint1-pop-quiz/backend  main  cat /tmp/sprint1-php-error.log                                  ✔ system
[15-Feb-2026 10:32:53 UTC] PHP Fatal error:  Uncaught Error: Call to undefined method Slim\Psr7\Stream::write() in /Users/mac/projects/sprint1-pop-quiz/ba
ckend/index.php:41
Stack trace:
#0 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/Handlers/Strategies/RequestResponse.php(39): {closure}(Object(Slim\Psr7\Request), Ob
ject(Slim\Psr7\Response), Array)
#1 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/Routing/Route.php(362): Slim\Handlers\Strategies\RequestResponse->__invoke(Object(Cl
osure), Object(Slim\Psr7\Request), Object(Slim\Psr7\Response), Array)
#2 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/MiddlewareDispatcher.php(73): Slim\Routing\Route->handle(Object(Slim\Psr7\Request))
#3 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/MiddlewareDispatcher.php(73): Slim\MiddlewareDispatcher->handle(Object(Slim\Psr7\Req
uest))
#4 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/Routing/Route.php(321): Slim\MiddlewareDispatcher->handle(Object(Slim\Psr7\Request))
#5 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/Routing/RouteRunner.php(74): Slim\Routing\Route->run(Object(Slim\Psr7\Request))
#6 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/MiddlewareDispatcher.php(73): Slim\Routing\RouteRunner->handle(Object(Slim\Psr7\Requ
est))
#7 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/App.php(209): Slim\MiddlewareDispatcher->handle(Object(Slim\Psr7\Request))
#8 /Users/mac/projects/sprint1-pop-quiz/backend/vendor/slim/slim/Slim/App.php(193): Slim\App->handle(Object(Slim\Psr7\Request))
#9 /Users/mac/projects/sprint1-pop-quiz/backend/index.php(105): Slim\App->run()
#10 {main}
  thrown in /Users/mac/projects/sprint1-pop-quiz/backend/index.php on line 41
```

Figure 1: error logs in the file `/tmp/sprint1-php-error.log`

  The error logs indicate that the method `Slim/Psr7/Stream::write()` is undefined. Upon rechecking the file, we found that the method name contained a hidden invisible character, causing PHP to treat it as an undefined method.

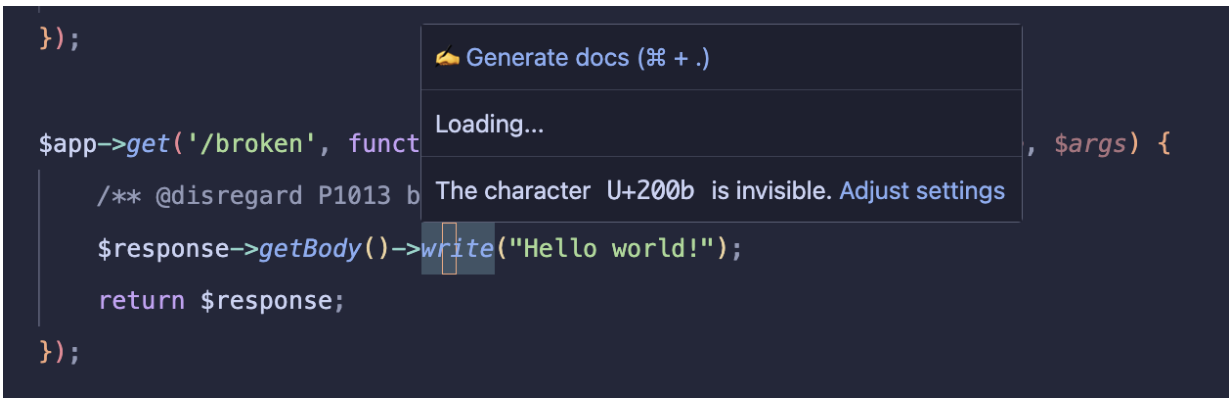


Figure 2: `write()` method call

- **Solution**

  Correcting the line to

```
    $response->getBody()->write("Hello world!");
```

  resolved the 500 Internal Server Error.

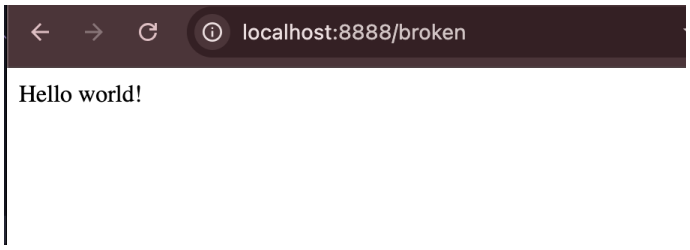


Figure 3: `/broken` endpoint

In addition to correcting the method call, we can facilitate debugging during local development by temporarily displaying PHP errors in the browser. This helps to quickly identify issues without checking log files manually. To enable this, we need to modify the error settings in `index.php`:

```
ini_set('display_errors', 1);
error_reporting(E_ALL);
```

1.2 Concurrent Connections Test (/crach) :

To test the behavior of the `/crash` endpoint under multiple simultaneous requests, we used the Apache Benchmark command:

```
ab -n 200 -c 10 http://localhost:8888/crash
```

Explanation of the command:

- `ab` is Apache Benchmark, a tool for testing HTTP server performance.
- `-n 200` specifies the total number of requests to perform (200 requests).
- `-c 10` sets the number of concurrent requests at a time (10 requests simultaneously).

```
~/projects/sprint1-pop-quiz/backend  main  ab -n 200 -c 10 http://localhost:8888/crash
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Finished 200 requests


Server Software:
Server Hostname:        localhost
Server Port:            8888

Document Path:          /crash
Document Length:        0 bytes

Concurrency Level:      10
Time taken for tests:   0.603 seconds
Complete requests:      200
Failed requests:        0
Non-2xx responses:      200
Total transferred:      70000 bytes
HTML transferred:       0 bytes
Requests per second:    331.79 [#/sec] (mean)
Time per request:       30.139 [ms] (mean)
Time per request:       3.014 [ms] (mean, across all concurrent requests)
Transfer rate:          113.41 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       0
Processing:     5   28   3.6     29      33
Waiting:        4   28   3.6     28      33
Total:          5   28   3.6     29      34

Percentage of the requests served within a certain time (ms)
  50%     29
  66%     29
  75%     30
  80%     31
  90%     32
  95%     33
  98%     33
  99%     33
 100%     34 (longest request)
```

Figure 4: inital result of the testing th /crash endpoint against concurrent connections

Observed Results:

- Total requests completed: 200
- Failed requests: 0 (no network errors)
- Non-2xx responses: 200 (all requests returned application-level errors)
- Requests per second: 331.79 [#/sec] (mean)
- Time per request (mean): 30.139 ms
- Total transfer rate: 113.41 Kbytes/sec

Analysis:
- The server handled connections successfully, but all responses were non-2xx, indicating that the application logic inside the `/crash` endpoint caused the errors.

Investigation :

Step 1: Initial Error – Missing Log File
We then check the log files to verify the source of the error.

```
  thrown in /Users/mac/projects/sprint1-pop-quiz/backend/index.php on line 27
[15-Feb-2026 12:28:19 UTC] PHP Warning:  file(/Users/mac/projects/sprint1-pop-quiz/backend/request_log.txt): Failed to open stream: No such file or direct
ory in /Users/mac/projects/sprint1-pop-quiz/backend/index.php on line 26
[15-Feb-2026 12:28:19 UTC] PHP Fatal error:  Uncaught TypeError: count(): Argument #1 ($value) must be of type Countable|array, false given in /Users/mac/
projects/sprint1-pop-quiz/backend/index.php:27
Stack trace:
```

- The file `request_log.txt` does not exist yet.
- Calling `count()` on the result of `file()` caused a fatal error because `file()` returned `false`.

Solution :
We have to ensure that the file exists or handle the case when it does not:

```
if (!file_exists($logFile)) {
    touch($logFile);
}
$logEntries = file($logFile);
```

This prevents fatal errors on the first request.

Step 2: Memory-Heavy Logging Error

Observation:
After fixing the log file issue, we encountered another problem when performing concurrent requests :

```
~/projects/sprint1-pop-quiz/backend  main  ab -n 200 -c 20 http://localhost:8888/crash
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Finished 200 requests


Server Software:
Server Hostname:        localhost
Server Port:            8888

Document Path:          /crash
Document Length:        12 bytes

Concurrency Level:      20
Time taken for tests:   0.554 seconds
Complete requests:      200
Failed requests:        189
   (Connect: 0, Receive: 0, Length: 189, Exceptions: 0)
Total transferred:      103943 bytes
HTML transferred:       37743 bytes
Requests per second:    360.85 [#/sec] (mean)
Time per request:       55.424 [ms] (mean)
Time per request:       2.771 [ms] (mean, across all concurrent requests)
Transfer rate:          183.15 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.2      0       1
Processing:     6   51   8.8     50      74
Waiting:        6   50   8.8     50      73
Total:          8   51   8.7     50      74
```

This log shows that 189 requests failed.
we then verify again the logs to inditify the source of the problem.

```
[15-Feb-2026 13:09:29 UTC] PHP Fatal error:  Allowed memory size of
   8388608 bytes exhausted (tried to allocate 10485776 bytes) in /
   Users/mac/projects/sprint1-pop-quiz/backend/index.php on line 33
```

The `logRequestWithRotation()` function attempts to allocate a very large string to clear the log when it exceeds 10 lines:

```
$contentClear = str_repeat('A', 0x9FFFF0);
file_put_contents($logFile, $contentClear);
```

With the PHP memory limit set to 8 MB:

```
ini_set('memory_limit', '8M');
```

this allocation exceeds available memory, especially when multiple requests happen concurrently.

Solution: We replace the memory-heavy clearing logic with a safe truncation approach:

```
function logRequestWithTruncation($message, $maxLines = 10)
{
    $logFile = __DIR__ . '/request_log.txt';
    $logEntry = "[" . date("Y-m-d H:i:s") . "] " . $message . PHP_EOL;

    if (!file_exists($logFile)) {
        touch($logFile);
    }

    // Read current log safely
    $logEntries = file($logFile, FILE_IGNORE_NEW_LINES |
       FILE_SKIP_EMPTY_LINES);

    // Keep only the last $maxLines - 1 entries
    $logEntries = array_slice($logEntries, -($maxLines - 1));

    // Overwrite the log with truncated entries
    file_put_contents($logFile, implode(PHP_EOL, $logEntries) . PHP_EOL);

    // Append the new log entry with exclusive lock
    file_put_contents($logFile, $logEntry, FILE_APPEND | LOCK_EX);
}
```

Advantages of this approach:

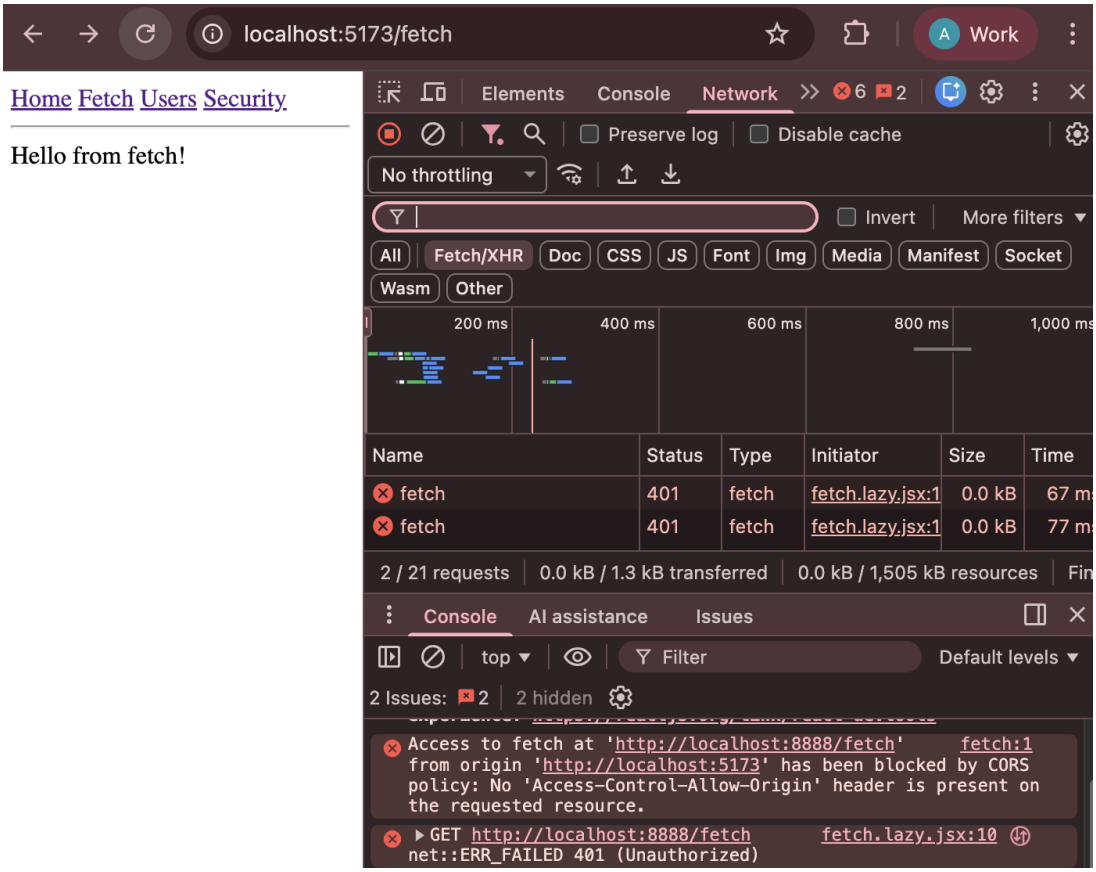
- **Memory-safe:** avoids allocating huge strings.
- **Crash-safe:** works even if the log file is missing.
- **Concurrency-safe:** `LOCK_EX` prevents simultaneous writes from overwriting each other.
- **History-preserving:** retains the last N log entries

2 Chapter 2: Challenges Frontend

2.1 Frontend Fetch/XHR Issue on /fetch endpoint :

Problem 1:

The React component calling the endpoint '/fetch' initially failed.



The cause of this failure is that :

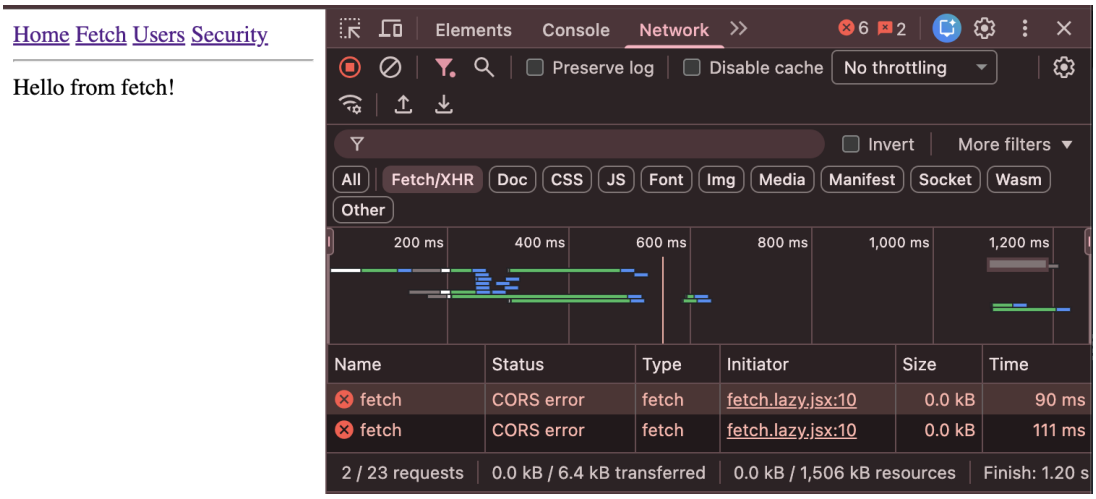
- Backend requires Basic Authentication for this route.
- Frontend fetch did not send the Authorization header.
- As a result, the server returned 401 Unauthorized for all requests.

Solution: We'll have to include the Authorization header in fetch:

```
fetch(`${import.meta.env.VITE_API_URL}/fetch`, {
  headers: {
    'Authorization': 'Basic ${import.meta.env.VITE_API_AUTH}'
  }
})
.then(response => response.json())
.then(data => console.log(data))
```

Problem 2:

Outcome: Requests to '/fetch', still fail due to a CORS (Cross-Origin Resource Sharing) error..



Cause:

- The browser sends a `OPTIONS` preflight request because the fetch includes custom headers (Authorization).
- The backend did not respond to the OPTIONS request with the required CORS headers.
- As a result, the browser blocked the actual GET request.

Solution:
We updated the Slim PHP backend with both CORS middleware and an OPTIONS route:

```
// Middleware to add CORS headers
$app->add(function ($request, $handler) {
    $response = $handler->handle($request);
    return $response
        ->withHeader('Access-Control-Allow-Origin', 'http://localhost:5173
            ')
        ->withHeader('Access-Control-Allow-Headers', 'Authorization,
            Content-Type')
        ->withHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS')
            ;
});

// Handle OPTIONS preflight requests
$app->options('/{routes:.+}', function ($request, $response) {
    return $response;
});
```

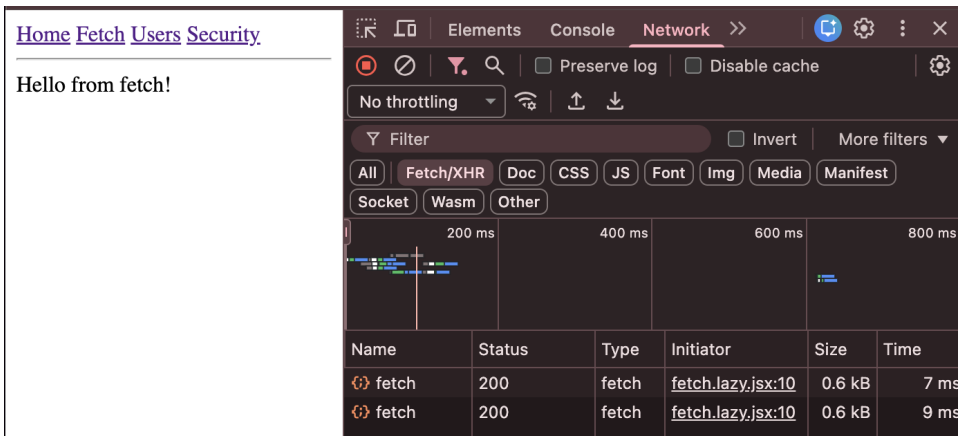


Figure 5: endpoint /fetch

2.2 XHR Issue on the /users Page

While testing the `/users` page, the frontend failed to retrieve data from the backend. The request was executed but returned an 405 response.

Frontend implementation:

```
useEffect(() => {
  fetch(`${import.meta.env.VITE_API_URL}/users`, {
    method: "POST",
  })
  .then(response => response.json())
  .then(data => console.log(data))
},[])
```

Backend route definition:

```
$app->any('/users', function (Request $request, Response $response, $args)
    {

    if ($request->getMethod() === 'POST') {
        $response->getBody()->write("Method Not Allowed");
        return $response->withStatus(405);
    }

    // returns JSON users list
});
```

Root Cause Analysis:

The issue occurred because the frontend was sending a **POST** request, while the backend explicitly rejects POST requests on this route by returning a **405 (Method Not Allowed)** status code.

After receiving this response, the frontend attempted to parse it as JSON using `response.json()`. However, the server returned plain text ("Method Not Allowed") instead of JSON, which caused a response format mismatch.

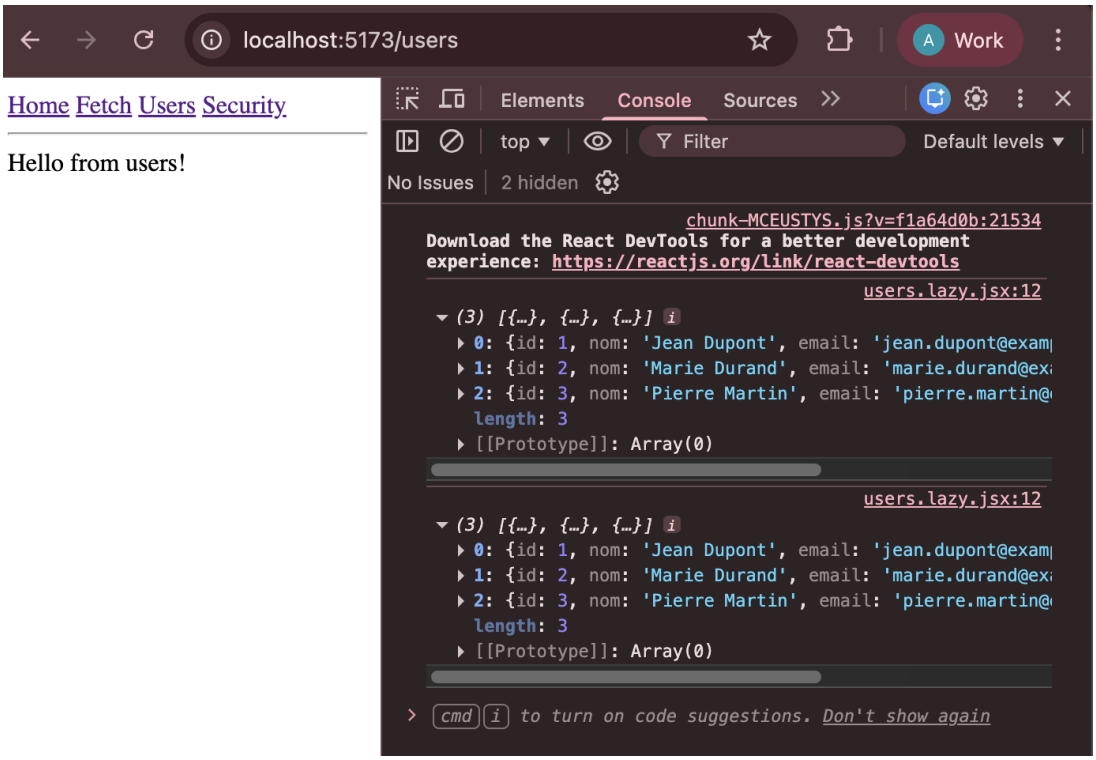
Since the `/users` endpoint is intended to retrieve a list of users, the correct HTTP method is **GET**.

Solution Implemented: The request method was changed from POST to GET.

```
useEffect(() => {
    fetch(`${import.meta.env.VITE_API_URL}/users`)
    .then(response => response.json())
    .then(data => console.log(data))
  },[])
```

Outcome:

After applying the correction, the `/users` endpoint successfully returns the expected JSON response, and the frontend properly displays the retrieved user data.



2.3 Asset Download Optimization

Problem Context:

The application is deployed regularly (daily) with frequent JavaScript code modifications. We need to optimize asset downloading to:

- Reduce loading times for users
- Ensure users always receive the latest version after deployment

Observation in Development Mode:

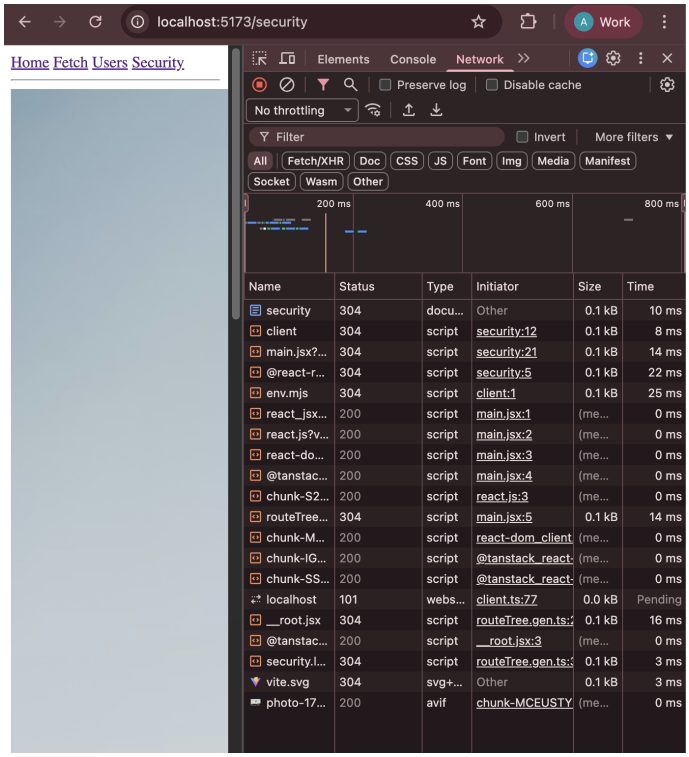


Figure 6: Assets re-downloaded on every reload in development mode

In development mode, we observe that all assets are re-downloaded on every page reload except some. However, **this is the expected and correct behavior**.

Why Cache is Disabled in Development:

Vite intentionally disables browser caching in development mode to:

- **Hot Module Replacement (HMR):** Code changes appear instantly
- **Avoid stale content:** Always see the latest code version

HTTP headers in dev mode:

```
Cache-Control: no-cache
```

Important: This behavior is desirable in development.

Difference Between Dev and Production Mode:

Aspect	Dev Mode	Production
Goal	Fast iteration	User performance
Cache	Disabled (intentional)	Enabled (optimized)
Filenames	Original	Hashed
Example	`main.jsx`	`main.a3f2b9e1.js`
HMR	Yes	No
Headers	`no-cache`	`max-age=31536000`

Table 1: Dev vs Prod Comparison

Solution: Production Optimization

The optimization strategy applies **only to production builds** via Vite configuration.

1. Optimized Vite Configuration

Modified `vite.config.js` file:

```
export default defineConfig({
  plugins: [react(), TanStackRouterVite(),],
  build: {
    rollupOptions: {
      output: {
        assetFileNames: assetInfo => {
          const ext = path.extname(assetInfo.name).substring(1);
          if (['woff', 'woff2', 'ttf', 'eot'].includes(ext)) {
            return 'assets/fonts/[name].[hash].[ext]';
          } else if (['png', 'jpg', 'jpeg', 'svg', 'gif'].includes(ext)) {
            return 'assets/images/[name].[hash].[ext]';
          } else if (['css'].includes(ext)) {
            return 'assets/css/[name].[hash].[ext]';
          } else {
            return 'assets/[name].[hash].[ext]';
          }
        },
        chunkFileNames: 'js/[name].[hash].js',
        entryFileNames: 'js/[name].[hash].js',
      },
    },
  }
})
```

After code modification and new build:

- Changed files get new hash: `index.NEW_HASH.js`
- Unchanged files keep same hash: `file.7d8e4f2c.js` (still cached)
- Only modified files are downloaded

2.4 Preventing XSS on /security page:

The /security page loaded an image from an external URL:

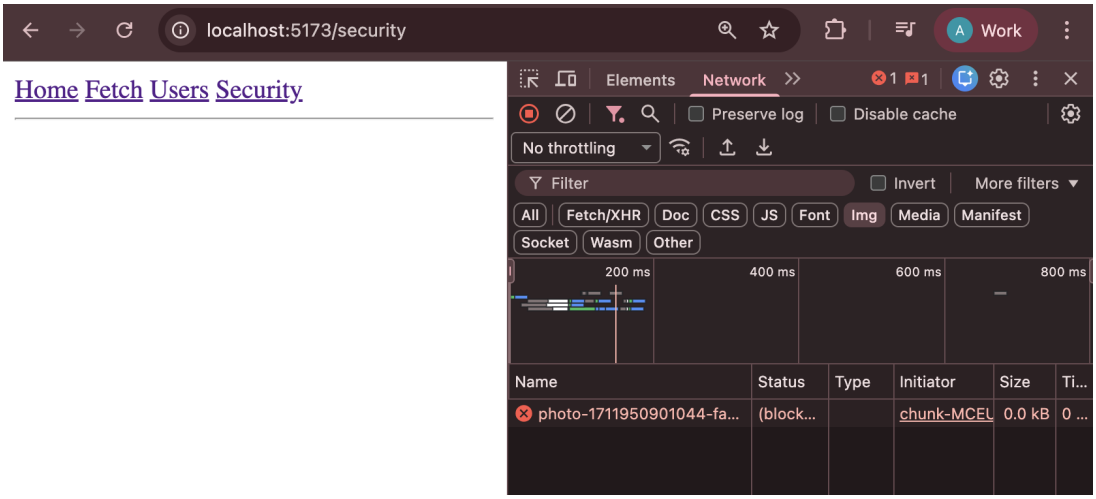
```
<img src="https://images.unsplash.com/photo-1711950901044-fa6215a9c59b?q
   =80&w=2970&auto=format&fit=crop" alt="" />
```

Problem: Loading external resources can introduce Cross-Site Scripting (XSS) vulnerabilities.

Solution: We implemented the CSP using a `<meta>` tag in the Vite project's `index.html`:

```
<meta http-equiv="Content-Security-Policy" content="img-src 'self';" />
```

Explanation:

- `img-src 'self';` restricts images to be loaded only from the same origin (localhost in dev mode).
- This prevents any external image from being loaded, mitigating potential XSS attacks.
- Using a `<meta>` tag is suitable for frontend-only assets during development.

Result:
After adding the meta tag, loading the /security page in the browser blocks the external Unsplash image, ensuring that only images from the local origin are allowed. This provides a lightweight and effective frontend-side protection for development environments.